

SatsPath

Funcionamiento actual, capacidades y camino hacia producción

Repositorio analizado: Truja503/satspath

Estado del análisis: julio de 2026

Objetivo del documento: explicar qué es SatsPath, cómo funciona actualmente, qué puede lograr con su implementación presente y qué componentes faltan para convertirlo en infraestructura utilizada por wallets y aplicaciones reales.

1. Resumen ejecutivo

SatsPath es un protocolo y conjunto de herramientas para descubrir, verificar y seleccionar métodos de pago de Bitcoin a partir de un identificador legible, por ejemplo:

`rodrigo@satspath.dev`

En lugar de obligar al pagador a conocer previamente si el receptor utiliza Lightning, una dirección on-chain, Ark, BOLT12 u otro método, SatsPath permite publicar un perfil firmado que contiene los métodos públicos disponibles.

El flujo general es:

```
Identificador
↓
Cadena de resolvers
↓
Perfil de pago firmado
↓
Verificación de firma, identidad y expiración
↓
Selección del método de pago
↓
Invoice, URI, QR o handoff hacia una wallet
```

SatsPath puede utilizar información pública real de Bitcoin y generar instrucciones que una wallet puede pagar. Por ejemplo, puede obtener una invoice BOLT11 mediante LNURL, construir un URI BIP-21 para una dirección on-chain o devolver un pointer de Ark.

SatsPath no necesita almacenar seeds ni claves privadas de fondos para cumplir su función. Las claves de identidad del protocolo sirven para firmar perfiles públicos y demostrar que estos no fueron modificados. Las claves que gastan Bitcoin permanecen dentro de la wallet del usuario.

La propuesta central puede resumirse así:

SatsPath transforma una identidad legible en una instrucción de pago verificable y selecciona el rail más apropiado para que una wallet complete el pago.

2. Problema que resuelve

Bitcoin ya no posee una sola forma de recibir pagos. Un usuario puede aceptar fondos mediante:

- Lightning Address.
- LNURL-pay.
- BOLT11.
- BOLT12.

- Dirección Bitcoin on-chain.
- Silent Payments.
- Ark.
- Otros sistemas compatibles o futuros adaptadores.

Cada método tiene diferencias en:

- Velocidad.
- Comisión.
- Privacidad.
- Disponibilidad.
- Liquidez.
- Interoperabilidad.
- Dependencias de infraestructura.

Actualmente, la mayoría de wallets obliga al usuario o desarrollador a conocer de antemano cuál método utilizar. SatsPath introduce una capa intermedia que responde tres preguntas:

1. ¿Quién es el receptor?
2. ¿Qué métodos públicos acepta?
3. ¿Cuál de esos métodos debería utilizarse para este pago?

3. Qué es un perfil de pago

El elemento principal del protocolo es el **SignedPaymentProfile**.

Un perfil puede incluir:

```
{
  "alias": "rodrigo@satspath.dev",
  "identity_pubkey": "02...",
  "methods": [
    {
      "type": "Lightning",
      "lightning_address": "rodrigo@satspath.dev"
    },
    {
      "type": "Onchain",
      "network": "Mainnet",
      "address": "bc1q..."
    },
    {
      "type": "Ark",
      "server": "https://ark.example.com",
      "pubkey": "02..."
    }
  ],
  "updated_at": 1782810000,
  "expires_at": 1785402000,
  "sequence": 4
}
```

El perfil se firma con una clave de identidad secp256k1.

```
profile
  ↓ canonical JSON
SHA-256
```

↓

Firma ECDSA secp256k1

La firma permite comprobar que:

- El perfil no fue modificado.
- Los métodos no fueron reemplazados durante la resolución.
- La clave pública de identidad corresponde con la firma.
- La versión recibida conserva integridad criptográfica.

La clave de identidad de SatsPath no tiene que controlar fondos. Su función es firmar información pública, no firmar transacciones de Bitcoin.

Implementación principal:

```
crates/satspath-core/src/profile.rs
crates/satspath-core/src/crypto.rs
crates/satspath-core/src/validation.rs
```

4. Funcionamiento completo

4.1 Creación de identidad

El usuario genera localmente una clave de identidad para SatsPath.

Esta clave permite:

- Firmar perfiles.
- Firmar invitaciones.
- Identificar actualizaciones legítimas.
- Rotar la identidad en versiones posteriores.

No sustituye las claves de una wallet ni debe utilizarse para controlar UTXOs o canales Lightning.

4.2 Registro de métodos de recepción

El receptor agrega uno o varios métodos públicos:

```
Lightning Address
Dirección on-chain
Silent Payment address o pubkey
BOLT12 offer
Ark server + pubkey o receive pointer
```

SatsPath valida el formato y construye un perfil público.

4.3 Firma del perfil

El perfil se serializa y firma con la clave de identidad.

El resultado es un objeto que puede publicarse por varios medios sin confiar plenamente en el transporte.

4.4 Publicación

El perfil firmado puede distribuirse mediante:

- Registro local.
- Endpoint HTTPS `.well-known`.
- BIP-353 mediante DNS.
- Nostr y NIP-05.

- Pear/Holepunch P2P.
- Exportación e importación manual.
- Un futuro resolver de plataforma.

La arquitectura es neutral al transporte. HTTPS, DNS, Nostr y P2P son formas de mover el perfil, pero la confianza final proviene de la firma y de las pruebas adicionales de identidad.

4.5 Resolución

El pagador introduce un identificador:

`alice@example.com`

SatsPath consulta una cadena de resolvers hasta encontrar un perfil.

La cadena actual puede incluir:

```

Registro local
  ↓
BIP-353
  ↓
HTTPS well-known
  ↓
Nostr / NIP-05
  ↓
Pear / Holepunch P2P

```

El resolver también compara el alias solicitado con el alias contenido en el perfil para evitar ataques de sustitución.

Implementación principal:

```

crates/satspath-core/src/resolver.rs
crates/satspath-core/src/resolvers/
crates/satspath-core/src/registry.rs
crates/satspath-core/src/peer_registry.rs

```

4.6 Verificación

Antes de utilizar el perfil, SatsPath puede comprobar:

- Firma criptográfica.
- Alias correcto.
- Expiración.
- Número de secuencia.
- Presencia de material privado prohibido.
- Pruebas de propiedad de métodos.
- Procedencia del resolver cuando esté disponible.
- Compatibilidad de red de una dirección Bitcoin.

Una firma válida demuestra control de la identidad de protocolo, pero no necesariamente control del correo electrónico o dominio utilizado en el alias. Esa segunda capa requiere DNSSEC, NIP-05, HTTPS verificado o un challenge de plataforma.

4.7 Selección de rail

Después de resolver y verificar el perfil, el router analiza los métodos disponibles.

Actualmente existen políticas para:

- Seleccionar Lightning en pagos pequeños cuando está disponible.

- Utilizar on-chain según las comisiones estimadas.
- Utilizar Ark como alternativa.
- Generar razones legibles para la decisión.
- Evaluar urgencia y snapshots de fees.
- Preparar rutas divididas en módulos experimentales.

Implementación principal:

```
crates/satspath-router/src/router.rs
crates/satspath-router/src/scoring.rs
crates/satspath-router/src/priority.rs
crates/satspath-router/src/fees.rs
```

4.8 Construcción de la instrucción de pago

Dependiendo del método elegido, SatsPath puede producir:

Lightning

- Lightning Address.
- LNURL.
- Invoice BOLT11 obtenida desde un callback LNURL-pay.
- BOLT12 offer o información relacionada.

On-chain

```
bitcoin:bc1q...?amount=0.00021000
```

Ark

```
ark:<pubkey>?server=<server>&amount=<sats>
```

También puede generar un QR SVG y una respuesta JSON estable para el frontend.

4.9 Pago real mediante una wallet

La instrucción producida por SatsPath puede ser entregada a una wallet que controle fondos.

SatsPath

↓ invoice / URI / pointer

Wallet del pagador

↓ firma y ejecución

Bitcoin, Lightning o Ark

Esto permite realizar pagos reales sin que SatsPath almacene las claves privadas de los fondos.

La responsabilidad se divide así:

Componente	Responsabilidad
SatsPath	Resolver identidad, verificar perfil, escoger rail y construir la instrucción
Wallet	Mostrar confirmación, firmar y ejecutar el pago
Red o protocolo	Liquidar o transmitir el pago

Esta separación es una propiedad del diseño, no una limitación conceptual. Mantiene a SatsPath como infraestructura no custodial y facilita su integración con varias wallets.

5. Qué existe actualmente

5.1 Core del protocolo

Existe implementación para:

- `PaymentProfile`.
- `SignedPaymentProfile`.
- Métodos Lightning, on-chain y Ark.
- Firma y verificación secp256k1.
- Validación de datos públicos.
- Invitaciones.
- Expiración.
- Nonces.
- Secuencias.
- Pruebas de propiedad.
- Rotación de claves.
- Codificación de requests universales.

5.2 Resolver chain

Existen resolvers para:

- Registro local.
- HTTPS.
- BIP-353.
- Nostr y NIP-05.
- P2P mediante el SDK Pear/Holepunch en el daemon.

El resolver HTTPS verifica firma y expiración antes de aceptar un perfil.

5.3 Router

El router puede:

- Detectar métodos disponibles.
- Consultar fees on-chain.
- Elegir un rail.
- Calcular una comisión aproximada.
- Devolver ETA y explicación.
- Construir un payload de pago.
- Producir un contrato `QuoteResponse` estable.

Estados actuales:

```
ok
not_registered
no_route
invalid_signature
```

5.4 LNURL y BOLT11

La implementación puede:

1. Resolver una Lightning Address.
2. Obtener metadata LNURL-pay.
3. Validar `minSendable` y `maxSendable`.
4. Solicitar una invoice.
5. Parsear BOLT11.

6. Verificar el monto.
7. Verificar expiración.
8. Entregar la invoice como payload o QR.

5.5 Daemon local

satspathd expone una API HTTP con endpoints para:

```
GET /health
GET /v1/status
GET /v1/node
GET /v1/profile
POST /v1/profile
POST /v1/profile/methods
POST /v1/resolve
POST /v1/quote
POST /v1/pay
POST /v1/send
POST /v1/receive
POST /v1/dns/resolve
```

El daemon también incluye una UI local, generación de QR, administración de perfiles y un bridge hacia el SDK P2P.

5.6 SDKs y bindings

El workspace contiene:

```
satspath-core
satspath-router
satspath-cli
satspath-swaps
satspathd
satspath-wasm
satspath-ffi
```

También existen bindings generados para Kotlin y Swift mediante UniFFI, además de funciones WASM para aplicaciones web.

5.7 P2P

El SDK de Pear/Holepunch permite publicar y resolver perfiles firmados mediante Hyperswarm.

El daemon puede iniciar procesos Node para:

- Publicar un perfil.
- Resolver perfiles.
- Validar el perfil nuevamente en Rust.
- Guardar perfiles resueltos en registros locales.

5.8 Funciones avanzadas presentes

El repositorio contiene módulos para:

- BOLT12.
- Silent Payments.
- Split Payments.
- Key Rotation.
- Ownership proofs.

- Ark intents y rutas.
- WASM.
- FFI móvil.

Algunas de estas funciones están implementadas a nivel de tipos, parsing o planificación, pero aún necesitan interoperabilidad completa con wallets reales.

6. Qué puede lograr SatsPath

6.1 Identidad de pago universal

Un usuario podría compartir una sola identidad:

`rodrigo@satspath.dev`

sin publicar manualmente una dirección diferente para cada protocolo.

6.2 Descubrimiento automático

Una wallet podría descubrir qué métodos acepta el receptor sin pedirle que seleccione manualmente Lightning, Ark u on-chain.

6.3 Selección inteligente

El router puede evolucionar para elegir según:

- Fee máximo.
- Urgencia.
- Privacidad.
- Liquidez Lightning.
- Salud del rail.
- Monto.
- Preferencias del usuario.
- Compatibilidad de la wallet.
- Confianza del resolver.

6.4 Interoperabilidad entre wallets

SatsPath puede convertirse en un SDK que varias wallets integren para obtener el mismo resultado a partir del mismo perfil firmado.

6.5 Comercios

Un comercio podría publicar un perfil con múltiples métodos y permitir que cada cliente pague mediante la ruta compatible con su wallet.

6.6 Fallback entre rails

Si Lightning no está disponible, la wallet podría utilizar Ark u on-chain sin solicitar otra dirección al receptor.

6.7 Infraestructura no custodial

SatsPath puede operar sin controlar fondos, reduciendo riesgos regulatorios y de seguridad asociados con custodiar Bitcoin.

6.8 Base para protocolos adicionales

El sistema puede agregar adaptadores para nuevos rails siempre que estos puedan representarse mediante instrucciones públicas verificables.

7. Qué hace falta

7.1 Verificación real de identidad

El daemon actual utiliza un verificador simulado para challenges de correo.

Hace falta implementar:

- Magic links reales.
- Tokens aleatorios de un solo uso.
- Hash del token en almacenamiento.
- Expiración.
- Rate limiting.
- Protección contra enumeración.
- Confirmación explícita del alias.
- Pruebas de control de dominio.

Para dominios propios se debe priorizar:

- DNSSEC y BIP-353.
- HTTPS `.well-known`.
- NIP-05.

Para correos de consumo se necesita un resolver de plataforma que valide la cuenta antes de publicar el perfil.

7.2 Procedencia y nivel de confianza

El resolver debería devolver no solamente el perfil, sino también su procedencia.

Propuesta:

```
struct ResolvedProfile {
    profile: SignedPaymentProfile,
    source: ResolverSource,
    identifier_verified: bool,
    verification_method: VerificationMethod,
    trust_level: TrustLevel,
    fetched_at: i64,
}
```

Posibles fuentes:

```
local
https
bip353_dnssec
nostr_nip05
p2p
platform
```

La aplicación necesita distinguir entre:

- Perfil firmado.
- Dominio verificado.
- Correo verificado.

- Método de pago verificado.
- Perfil recibido desde una fuente no autenticada.

7.3 Estados de error más precisos

Actualmente varios errores de resolución pueden terminar representados como `not_registered`.

Se recomienda ampliar el contrato:

```
ok
not_found
temporarily_unavailable
invalid_signature
expired_profile
identifier_unverified
unsupported_method
no_route
resolver_error
```

Esto evitará confundir una caída temporal de red con un usuario que no existe.

7.4 Seguridad del daemon

Antes de exponer `satspathd` fuera de localhost hacen falta:

- Autenticación de administración.
- Token de sesión local.
- Protección CSRF.
- CORS restringido.
- Límites de requests.
- Límite de tamaño de payload.
- Separación entre API pública y API administrativa.
- Restricción explícita para binds públicos.
- Logs sin datos sensibles.
- Manejo seguro de procesos P2P secundarios.

7.5 Protección de la clave de identidad

La clave de identidad no controla fondos, pero sí puede firmar perfiles falsos si es robada.

Se recomienda integrar:

- macOS Keychain.
- iOS Keychain.
- Android Keystore.
- Linux Secret Service.
- Windows Credential Manager.
- Cifrado AES-GCM como fallback.

También hace falta:

- Backup.
- Recuperación.
- Revocación.
- Rotación asistida.
- Confirmación en un segundo dispositivo.

7.6 Router basado en capacidades reales

La política actual utiliza umbrales generales. Para producción debe evolucionar hacia un sistema de scoring configurable.

Factores recomendados:

Costo estimado
Probabilidad de éxito
Tiempo esperado
Privacidad
Monto mínimo y máximo
Liquidez disponible
Disponibilidad del servidor
Confianza del método
Compatibilidad de la wallet
Urgencia
Preferencias del usuario

El router también debe respetar realmente:

max_fee_sats
max_fee_percent
preferred_rails
blocked_rails
privacy_mode

7.7 Disponibilidad real de Lightning

Hace falta reemplazar varias suposiciones estáticas por comprobaciones reales:

- Límites de LNURL.
- Capacidad de obtener invoice.
- Validez del callback.
- Expiración.
- Compatibilidad de red.
- Soporte de BOLT12.
- Resultado del intento de pago reportado por la wallet.

SatsPath no tiene que inspeccionar canales privados, pero la wallet integradora puede aportar señales sobre probabilidad de éxito y liquidez.

7.8 Protección de red y SSRF

Los resolvers HTTP, Nostr y LNURL deben aplicar políticas estrictas:

- HTTPS obligatorio salvo desarrollo local.
- Bloqueo de localhost y redes privadas para contenido remoto.
- Control de redirects.
- Timeouts.
- Límite de tamaño de respuesta.
- Validación del content type.
- Allowlist o reglas de dominio para callbacks.
- Resolución DNS segura.
- Protección contra rebinding.

7.9 BIP-353 y DNSSEC operativos

Hace falta convertir el soporte actual en una experiencia operativa completa:

- Resolver DNSSEC confiable.
- Publicación automatizada.
- Compatibilidad con proveedores DNS.
- Manejo de TTL.
- Rotación de registros.
- Validación de redes.
- Herramienta de diagnóstico.

7.10 Publicación Nostr desde las librerías principales

El resolver Nostr ya puede consumir perfiles, pero la publicación necesita una interfaz integrada y coherente para:

- Crear evento.
- Firmarlo.
- Publicar en varios relays.
- Confirmar recepción.
- Actualizar perfiles.
- Eliminar o invalidar versiones antiguas.

7.11 P2P embebido

El daemon posee un bridge funcional mediante procesos Node, pero el binding FFI todavía mantiene un placeholder para P2P embebido.

Hace falta decidir una dirección:

1. Mantener Hyperswarm como sidecar.
2. Integrar un runtime embebido.
3. Implementar un `P2pTransport` abstracto con múltiples backends.

La API debe evitar depender de rutas relativas del repositorio o scripts locales cuando se distribuya como aplicación instalada.

7.12 Interoperabilidad Ark

Para considerar Ark una integración completa hacen falta pruebas con una wallet y servidor concretos.

Se debe comprobar:

- Formato del receive pointer.
- URI aceptado.
- QR compatible.
- Expiración.
- Servidor correcto.
- Pubkey correcta.
- Flujo Ark-to-Ark.
- Fallback hacia Lightning u on-chain.
- Manejo de errores.

No se deben inventar APIs de Arkade ni asumir formatos sin una prueba de interoperabilidad.

7.13 BOLT12 completo

Hace falta validar el flujo completo:

Offer
↓
Invoice request
↓
Invoice
↓
Verificación
↓
Handoff a wallet

El parser por sí solo no es suficiente. Se necesitan test vectors y pruebas con implementaciones Lightning existentes.

7.14 Silent Payments

Para integración completa hacen falta:

- Generación y validación compatible con BIP-352.
- Network checks.
- Payment URI correcto.
- Compatibilidad con una wallet que pueda pagar.
- Manejo de scan keys sin exponer material privado.
- Test vectors oficiales.

7.15 Split Payments

El módulo debe definir claramente si divide:

- Un monto entre varios receptores.
- Un pago entre varios rails.
- Ingresos mediante políticas de porcentaje.
- Pagos secuenciales con tolerancia parcial a fallos.

También hacen falta:

- Atomicidad o definición explícita de no atomicidad.
- Idempotencia.
- Reintentos.
- Estado por receptor.
- Política de redondeo.
- Recuperación ante pagos parcialmente completados.

7.16 Revocación y rotación

La rotación de claves necesita integrarse con los resolvers.

Un cliente debe poder saber:

- Que una clave anterior autorizó la nueva.
- Cuál es la secuencia más reciente.
- Si una clave fue revocada.
- Si un perfil es un replay antiguo.
- Qué fuente confirma la transición.

7.17 Caché y replay protection

Cada perfil debería utilizar:

- `sequence` monotónica.

- `updated_at`.
- `expires_at`.
- Nonce.
- Clave efectiva después de rotación.

Los resolvers y caches deben rechazar versiones más antiguas que una versión previamente observada.

7.18 Observabilidad

Hace falta instrumentar:

- Tiempo de resolución.
- Resolver elegido.
- Resolver fallido.
- Perfil inválido.
- Rail elegido.
- Tiempo para obtener invoice.
- Fallo de handoff.
- Tasa de fallback.

Las métricas deben evitar almacenar alias, direcciones o invoices completas.

7.19 CI y política de releases

El repositorio ya contiene un workflow de CI, pero debe convertirse en un gate obligatorio.

Requisitos recomendados:

```
cargo fmt --all -- --check
cargo build --workspace
cargo test --workspace --all-targets
cargo clippy --workspace --all-targets -- -D warnings
cargo audit
cargo deny
Tests WASM
Tests FFI
Tests del SDK JavaScript
```

También hacen falta:

- Branch protection.
- Reviews obligatorias.
- PRs pequeños.
- Checklist con evidencia.
- Releases versionadas.
- Changelog.
- Artefactos firmados.
- Builds reproducibles.

7.20 Auditoría y fuzzing

Antes de una adopción amplia se recomienda:

- Auditoría externa de criptografía y parsing.
- Fuzzing de perfiles.
- Fuzzing de URIs.
- Fuzzing de respuestas HTTP y Nostr.
- Fuzzing de BOLT11 y BOLT12.

- Pruebas de perfiles gigantes.
- Pruebas de Unicode y normalización.
- Pruebas de colisión y canonicalización de alias.

8. Arquitectura recomendada para producción

Wallet o aplicación cliente

SatsPath SDK

- Resolver chain
- Verificación
- Trust model
- Router y scoring
- Payload builder

Resolvers

DNS / HTTPS
Nostr / P2P
Platform

Wallet adapter

Lightning
Bitcoin on-chain
Ark

La integración ideal mantiene una separación estricta:

SatsPath Core

Responsable de:

- Tipos.
- Firmas.
- Verificación.
- Resolución.
- Routing.
- Construcción de instrucciones.

Wallet Adapter

Responsable de:

- Obtener balance.
- Consultar capacidades.
- Confirmar con el usuario.
- Pagar invoice.
- Crear y firmar transacción.
- Reportar éxito o fallo.

Transport Adapter

Responsable de:

- DNS.
 - HTTPS.
 - Nostr.
 - P2P.
 - Plataforma.
-

9. Primera versión de producción recomendada

La primera versión no necesita soportar todas las funciones del repositorio.

Debe enfocarse en:

Resolve

Verify

Route

Generate payable instruction

Open external wallet

Alcance recomendado

- Lightning Address y LNURL-pay.
- Invoice BOLT11 validada.
- Dirección on-chain y BIP-21.
- BIP-353 para dominios controlados.
- HTTPS .well-known.
- Perfil firmado y expiración.
- QR.
- Wallet handoff.
- Ark como adapter experimental interoperable.

Fuera del primer alcance

- Split payments complejos.
- Ejecución automática multirail.
- Recuperación social.
- Registro global propio.
- Todos los proveedores DNS.
- Todos los runtimes P2P.

Reducir el alcance no disminuye la visión. Evita que una arquitectura prometedora se convierta en una colección de módulos a medio terminar, que es el destino natural de demasiados proyectos técnicamente ambiciosos.

10. Criterios de aceptación

SatsPath debería considerarse listo para una integración productiva inicial cuando cumpla:

Seguridad

- ☐ Los perfiles inválidos son rechazados.
- ☐ Los perfiles expirados son rechazados.

- ☐ Los replays con menor secuencia son rechazados.
- ☐ Las claves de identidad están protegidas por el sistema operativo o cifrado.
- ☐ La API administrativa requiere autenticación.
- ☐ No existe CORS abierto en una instancia pública.
- ☐ Los fetches remotos tienen protección SSRF.

Identidad

- ☐ La procedencia del perfil se incluye en la respuesta.
- ☐ El cliente distingue firma de perfil e identidad verificada.
- ☐ BIP-353 funciona con DNSSEC.
- ☐ Los challenges de plataforma son reales.

Pagos

- ☐ Una invoice LNURL válida puede abrirse y pagarse desde una wallet externa.
- ☐ Una invoice con monto incorrecto es rechazada.
- ☐ Una invoice expirada es rechazada.
- ☐ El URI BIP-21 es compatible con al menos dos wallets.
- ☐ El adapter Ark ha sido probado con una implementación concreta.

Integración

- ☐ Existe una API estable y versionada.
- ☐ Existe una librería usable en al menos una plataforma cliente.
- ☐ Hay test vectors públicos.
- ☐ Existe una aplicación demo que utiliza el SDK, no lógica duplicada.

Operaciones

- ☐ CI es obligatorio.
- ☐ Releases están firmadas.
- ☐ Existe changelog.
- ☐ Hay métricas sin información sensible.
- ☐ Existe política de vulnerabilidades.

11. Roadmap sugerido

Fase 1: consolidación del core

- Limpiar documentación contradictoria.
- Cerrar o actualizar issues obsoletos.
- Convertir CI en requisito obligatorio.
- Definir contrato `ResolvedProfile`.
- Mejorar estados de error.
- Aplicar políticas de replay y secuencia.

Fase 2: SatsPath Preview

- Perfil real publicado en HTTPS.
- BIP-353 funcional.
- LNURL y BOLT11 interoperables.
- BIP-21 interoperable.
- QR y wallet handoff.
- PWA basada en el SDK.

Fase 3: seguridad operativa

- Protección de claves.
- Auth del daemon.
- CORS seguro.
- SSRF protection.
- Rate limiting.
- Auditoría de dependencias.

Fase 4: integración de wallet

- Elegir una wallet de referencia.
- Integrar WASM, FFI o Rust directamente.
- Aportar capacidades de la wallet al router.
- Reportar resultado del pago.
- Implementar fallback después de un fallo.

Fase 5: Ark y rails adicionales

- Ark interoperable.
- BOLT12 completo.
- Silent Payments.
- Split Payments con semántica definida.

Fase 6: especificación abierta

- Congelar SatsPath Protocol v1.
 - Publicar test vectors.
 - Crear suite de conformidad.
 - Implementar un segundo cliente independiente.
 - Solicitar revisión externa.
-

12. Posicionamiento recomendado

SatsPath debería presentarse como:

Un protocolo abierto y no custodial para resolver identidades de pago Bitcoin, verificar perfiles firmados y seleccionar la mejor instrucción disponible entre múltiples rails.

No debería limitarse a describirse como:

- Una wallet.
- Un explorador de direcciones.
- Un registro de usuarios.
- Un router basado solamente en fees.
- Un sistema P2P específico.

Su valor está en unir cuatro capas:

Identidad
Verificación
Descubrimiento
Routing

13. Conclusión

SatsPath ya posee una base funcional considerable:

- Perfiles firmados.
- Validación criptográfica.
- Resolución multitransporte.
- LNURL y BOLT11.
- BIP-353.
- Nostr.
- P2P.
- Router.
- QR y wallet handoff.
- WASM y bindings móviles.

También puede facilitar pagos reales porque trabaja con información pública válida y produce instrucciones pagables por una wallet. No necesita custodiar fondos para tener utilidad práctica.

El trabajo restante no consiste principalmente en agregar más protocolos. Consiste en endurecer la seguridad, demostrar interoperabilidad y convertir las piezas actuales en una experiencia estable que otra wallet pueda integrar sin depender de supuestos internos del repositorio.

El punto que marcará la transición de proyecto a infraestructura será el siguiente:

Dos wallets independientes resuelven el mismo identificador, verifican el mismo perfil, seleccionan una ruta compatible y completan el pago utilizando la instrucción producida por SatsPath.

Cuando ese flujo sea reproducible, versionado y seguro, SatsPath dejará de ser únicamente una implementación prometedora y comenzará a funcionar como un protocolo de interoperabilidad real para pagos Bitcoin.

14. Referencias dentro del repositorio

```
README.md
docs/protocol.md
docs/resolvers.md
docs/implementations.md
docs/threat_model.md
docs/wire_p2p.md
crates/satspath-core/src/
crates/satspath-router/src/
crates/satspath-cli/src/
crates/satspathd/src/main.rs
crates/satspath-wasm/src/
crates/satspath-ffi/src/
sdk/satspath-p2p/
.github/workflows/ci.yml
```